

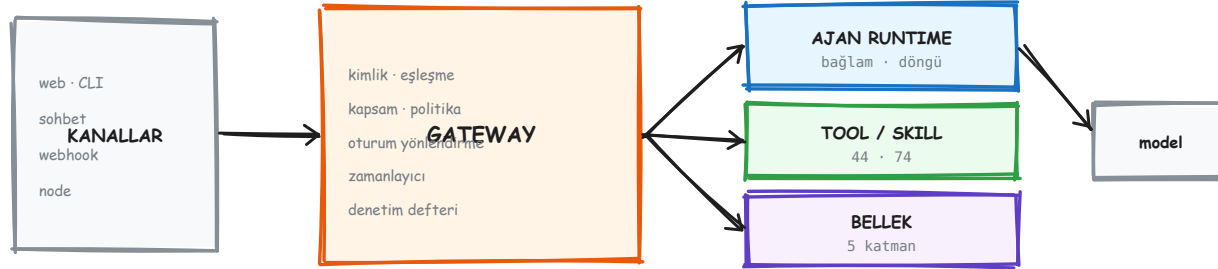
OpenClaw, on sekiz şemada

Ajanı kuşatan kontrol düzlemi: yetki, onay, denetim, bellek, zamanlama ve yeniden başlatma dayanıklılığı.

[@01cc7106](https://github.com/openclaw/openclaw)

uzun anlatım: [docs/pdf/openclaw-ici.pdf](#) · [docs/16](#) · [docs/18](#)

Kuşbakış1

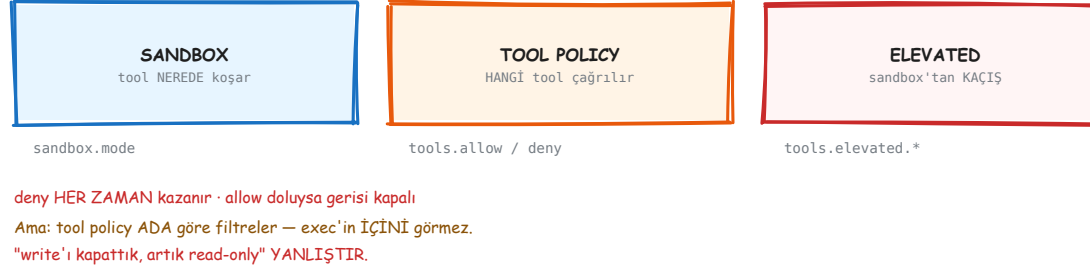


Her şey Gateway'den geçiyor: kimlik, yetki, kayıt ve zamanlama tek yerde. Ajan runtime bunların hiçbirini bilmiyor.

her şey Gateway'den geçer

Solda kanallar var: web arayüzü, komut satırı, sohbet uygulamaları, webhook'lar, cihaz düğümleri. Sağda yetenekler var: ajan runtime'ı, tool'lar ve skill'ler, bellek. Ortada **Gateway** duruyor, ve içinde kim olduğun, neye yetkili olduğun, hangi oturuma düştüğün, ne zamanlandığı ve ne kaydedildiği var. Mimarının özü tek cümlede: **ajan runtime'ı bunların hiçbirini bilmiyor**. Kimlik, yetki ve denetim ajan döngüsünün *dışında* kalıyor. Ve tam bu yüzden değiştirilebilir. Ajan motorunu söküp yerine başkasını koyabilirsin; kontrol düzlemi yerinde kalır.

Üç kontrol eksenı



sandbox: nerede · tool policy: hangisi · elevated: kaçış

"İzin" tek bir kavram değil, üç ayrı soru. Kodun *nerede* koştuğu, *hangi* tool'un çağrılabilirdiği, ve kutunun *dışına* çıkmanın bir yolu olup olmadığı. Bu üçünü karıştırmak en yaygın yapılandırma hatası. Filtre kuralı basit: `deny` her zaman kazanıyor, ve `allow` listesi doluysa listede olmayan her şey bloklu sayılıyor. Ama asıl bilinmesi gereken şu: **tool politikası tool'u yalnız adına göre filtreliyor**. Bir `exec` çağrısının içinde ne yapıldığına bakmıyor. Yani "write tool'unu kapattık, artık salt-okunur" cümlesi yanlıştır. `exec` serbestse kabuk üstünden yazmak zaten mümkün.

Metot kapsamı yalnızca ilk kapı



Yetki yükseltme yasak: bir cihazı onaylamak yalnız çağırının ZATEN sahip olduğu kapsamları basabilir.

Ve sessiz genişleme yok — daha geniş rol isteyen yeniden bağlanma, yeni bir bekleyen talep doğurur.

kapsam çağrının PARAMETRESİNDEN türetiliyor

Sekiz yetki kapsamı var, ama asıl fikir tabloda değil. Fikir şu: **aynı metot, farklı parametreyle farklı yetki istiyor.** `agent` metodu sıradan bir tur için yazma yetkisiyle geçiyor, ama `/reset` için yönetici istiyor. `node.invoke` normal komutlar için yazma, `browser.proxy` için yönetici. İki kural bunu tamamlıyor. **Yetki yükseltme yasak:** bir cihazı onaylarken yalnız zaten sahip olduğun kapsamları verebiliyorsun. Ve **sessiz genişleme yok:** daha geniş bir rol isteyen yeniden bağlanma otomatik geçmiyor, yeni bir onay talebi doğuruyor.

Onay komuta değil, plana bağlanır



Dosya onaydan sonra değişirse de reddedilir — kaymış içerik koşmaz.
Bizim approval.py bugün argüman hash'i TUTMUYOR. Açık iş.

argüman değiştiyse → approval mismatch

Naif bir onay akışında onay ile çalıştırma arasında bir boşluk kalıyor: kullanıcı gördüğü şeyi onaylıyor, ama çalışan başka bir şey olabiliyor. Aradan geçen sürede argümanlar değişmiş olabilir. OpenClaw onay isteğinin içine **kanonik bir plan** koyuyor: çalışma dizini, tam argüman listesi, sabitlenmiş dosya yolu. Onaylandıktan sonra *saklanan planı* çalıştırıyor, çağırının sonradan gönderdiğini değil. Bir dosyaya bağlıysa ve dosya onaydan sonra değiştiyse, kaymış içeriği çalıştırmak yerine koşuyu **reddediyor**. Yani onayladığın şey bir cümle değil, **donmuş bir plan**.

Dış içerik veri, talimat değil



<<<EXTERNAL_UNTRUSTED_CONTENT id="a3f9...">>>

id rastgele ÇÜNKÜ sabit olsaydı içerik kendi kapanışını yazıp sarmalayıcıdan çıkardı.

rastgele id'li sınır + 22 token temizliği + 28 homoglif katlaması

Modelin bağlamına giren her şey aynı görünüyor: metin. Senin sistem talimatın da metin, müşterinin gönderdiği PDF de metin. O PDF'in içinde "önceki talimatları yok say" yazıyorsa, model bunu neden talimat saymasın? Ayırt edecek bir işaret yok. OpenClaw dış içeriği bir sarmalayıcının içine koyuyor, ve sarmalayıcının id'si **rastgele**. Sabit olsaydı içerik kendi kapanış etiketini yazıp kutudan çıkardı. Ve şüpheli desenler yalnızca **loglanıyor**, engellenmiyor. Sebebi dürüst: desen eşleştirmeyeyle injection engellenemez. Tespit bir sinyal; asıl savunma sarmalayıcının kendisi.

Kademeli açığa çıkarma

PROMPT'TA NE VAR



gövde diskte bekler:



...74 tane

Gövde yalnız model `read` ile isteyince yüklenir — kademeli açığa çıkarma.

TAMAMI prompt'ta olsaydı

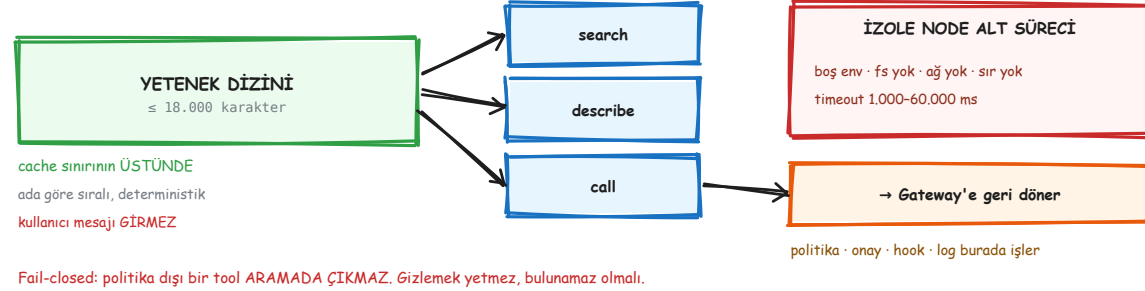
ölçülen fark:

%93 token tasarrufu

prompt'ta indeks, gövde talep üzerine → %93 tasarruf

Kurulu 74 skill var, ama prompt'a hepsinin gövdesi girmiyor. Yalnız **indeksleri** giriyor: ad ve tek satırlık açıklama. Model bir skill'e ihtiyaç duyduğunda gövdesini `read` ile çekiyor. İhtiyaç duymadıklarının gövdesi hiç yüklenmiyor. Kazanç yalnız token değil, aynı zamanda **isabet**. Model 74 talimatı aynı anda görmediği için dikkati dağılmıyor; yalnız işine yarayanı okuyor. Ve aynı fikir tool katmanında da *bağımsız olarak* ortaya çıkmış. Yani bu tek seferlik bir numara değil, tekrar eden bir **desen**.

Tool Search – büyük katalog, küçük prompt



dizin cache sınırının üstünde · köprü izole · fail-closed

Model bütün tool şemalarını görmüyor. Sınırlı bir yetenek dizini görüyor ve `search` → `describe` → `call` sırasını izliyor. Dizin ada göre sıralı, ve **cache sınırının üstüne** konmuş. Yani kullanıcı mesajının içine girmiyor. Girseydi her turda değişir ve prompt cache'i her turda bozulurdu. Aradaki kod köprüsü izole bir alt süreçte koşuyor: environment boş, dosya sistemi yok, ağ yok, **sır yok**. Köprüdeki kod tek başına hiçbir şey yapamıyor; her gerçek çağrı Gateway'e geri dönüyor ve normal politika, onay ve log yolundan geçiyor. Ve fail-closed: politikaya takılan bir tool **aramada hiç görünmüyor**.

Beş katman

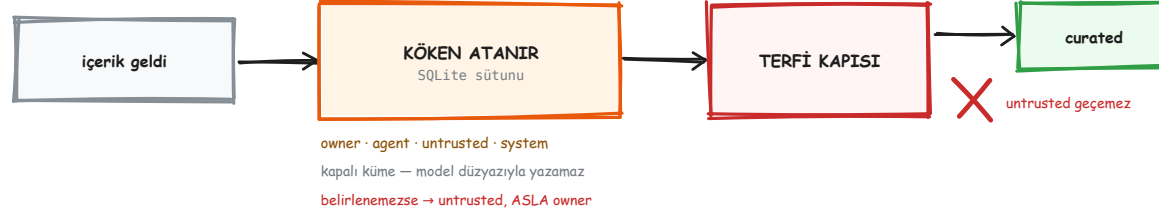
KATMAN	YÜZEY	KİM YAZAR	NE ZAMAN ENJEKTE
Instructions	AGENTS.md	yalnız insan yazar	her oturum
Curated core	MEMORY.md	kapılı konsolidasyon	her oturum
Episodic	memory/GG.md	ajan çalışırken	hiç — aranır
Prospective	SQLite	intent tool	tetiklenince
Review	DREAMS.md	dreaming	hiç — insan okur

Sınır ikinci ile üçüncü arasında: episodic'ten curated'a hiçbir şey kapıdan geçmeden çıkmaz.

asıl sınır ikinci ile üçüncü katmanın arasında

Beş katman var ama hepsini ezberlemene gerek yok; bilmen gereken tek şey sınırın nerede olduğu. **Curated** katmanı küçük, her oturumda bağlama giriyor, ve oraya yalnızca kapılı bir konsolidasyondan geçerek yazılabiliyor. **Episodic** katmanı büyük, ekleme dostu, ve **hiç enjekte edilmiyor**; yalnız arama yoluyla erişilebiliyor. Kural şu: episodic'ten curated'a hiçbir şey **kapıdan geçmeden** çıkmıyor. Yani büyük ve denetimsiz olan taraf, küçük ve her zaman okunan tarafa kendiliğinden sızamıyor. En üstteki katmanı yalnız insan yazıyor, en alttakini yalnız insan okuyor.

Güvenlik sınırı yazma yolunda



İki hijyen kuralı: cron/heartbeat/alt-ajan oturumları aday ÜRETMEZ · bellekten enjekte edilen içerik yeniden ÇIKARILMAZ.

"Yüz kez hatırlanan bir olgu tek bir olgu olarak kalır."

köken şemada zorunlu, kapalı küme, model yazamıyor

Belleğe zehirli bir olgu girdikten sonra onu içerik taramasıyla yakalamak güvenilir değil. "Şirketin CEO'su X" cümlesi doğru mu yanlış mı, metne bakarak anlaşılmıyor. Bu yüzden savunma "kötü belleği sonradan bul" değil, **kötü belleğin terfi edememesi** üstüne kurulmuş. Her kaydın bir **köken** sınıfı var. Bu sınıf kapalı bir kümeden seçiliyor ve SQLite'ta ayrı bir sütunda duruyor, yani modelin düzyazıyla yazamayacağı bir yerde. Sınıflandırma da muhafazakâr: kökeni belirlenemeyen dışsal bir içerik **untrusted** sayılıyor, **asla owner varsayılmıyor**. Ve döngü kırılıyor: bellekten enjekte edilen içerik, yeni bellek olarak geri yazılmıyor.

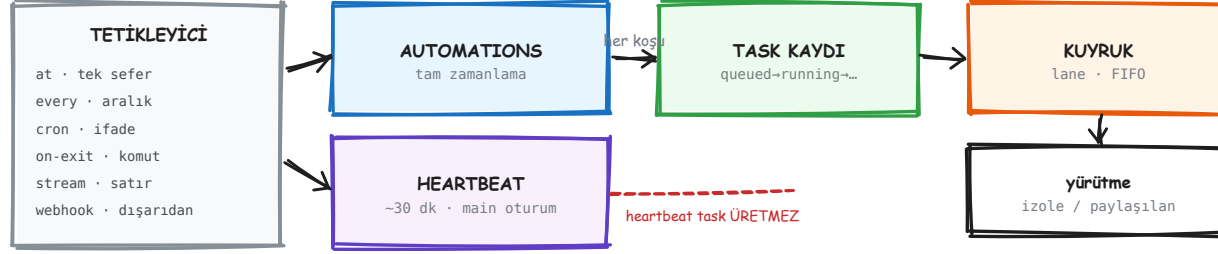
Bağlam motoru ve sıkıştırma



dört yaşam noktası, bir bozulmaz kural

"Modele ne gönderilecek" kararı çekirdeğe gömülü değil. Dört ayrı anda eklentiyle değiştirilebiliyor: mesaj eklenirken, model koşusundan hemen önce, pencere dolduğunda, ve tur bittiğinde. Altta pazarlığa açık olmayan tek bir kural duruyor: **tool çağırısı sonucundan ayrılmıyor**. Sebebi teknik ve kesin. Ayrılırdı model, cevabını hiç görmediği bir çağrı yapmış gibi görünürdü — ve çoğu sağlayıcı bu şekli geçersiz sayıp isteği **reddeder**. Yani sıkıştırma hatası bir kalite sorunu olarak değil, bir API hatası olarak karşına çıkıyor. Kendi sıkıştırıcını yazacaksan ilk yazacağın test bu.

Zamanlama yığını

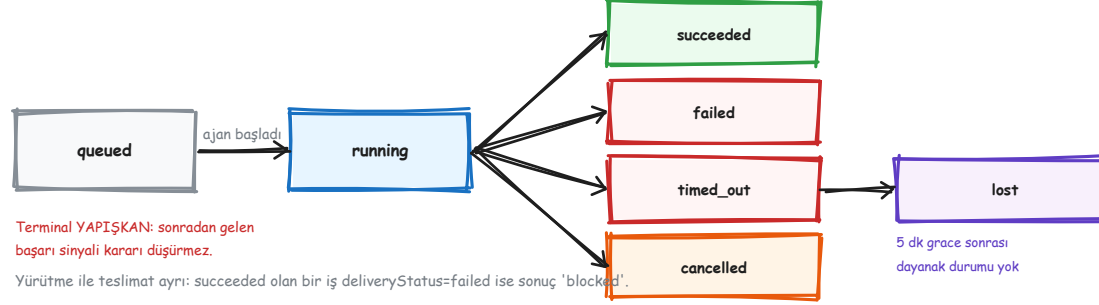


Sıra soldan sağa: ne tetikler → kim karar verir → ne kaydedilir → nasıl serileşir → nerede koşar.

ne tetikler → kim karar verir → ne kaydedilir → nasıl serileşir

"Arka planda iş çalıştır" tek bir ihtiyaç değil, ve OpenClaw bunu dört ayrı soruya bölmüş. Beş tetikleyici türü var: `at`, `every`, `cron`, `on-exit`, `stream`. Son ikisi zamana **hiç** bakmıyor; onlar olay kaynağı. Karar iki ayrı zamanlayıcıya bölünmüş. **Automations** tam zamanında koşuyor, izole bir oturum kullanıyor, ve her koşuda bir task kaydı üretiyor. **Heartbeat** yaklaşık zamanlı, tam bağlamlı, ve **hiç** kayıt üretmiyor. Ayrımın sebebi pratik: "her sabah 9'da rapor" izolasyon istiyor, "ara sıra gelen kutusuna göz at" ise bağlam istiyor.

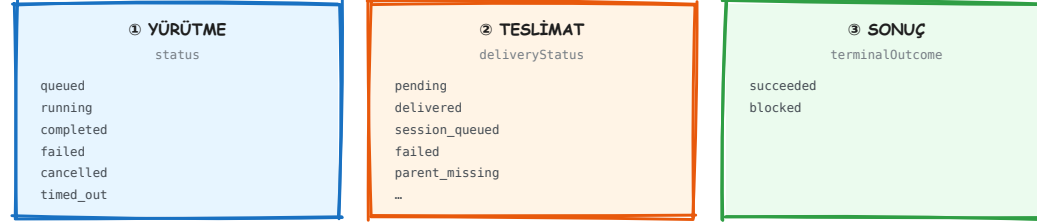
Task defteri – kayıt, zamanlayıcı değil



terminal durum yapışkan · lost bir kanıt standardı

Automations ve heartbeat işin *ne zaman* koşacağına karar veriyor. Task defteri ise *ne olduğunu* izliyor. İkisi ayrı bileşen, ve ayrı kalmaları önemli. Zamanlayıcı bir operasyon bileşeni; hızlı olmalı. Defter bir kanıt bileşeni; doğru olmalı. Karıştırırsan denetime götüreceğin kayıt, aynı zamanda performans için budanmış olan kayıt olur. **Terminal durum yapışkan**: bir iş iptal edildikten sonra gelen başarı sinyali operatörün kararını geri alamıyor. Ve **lost** için her kaynağın kendi kanıt standardı var. Kılavuzun cümlesi şu: "çevrimdışı bir denetim, kendi boş durumunu otorite saymaz."

Üç eksen, tip düzeyinde ayrı



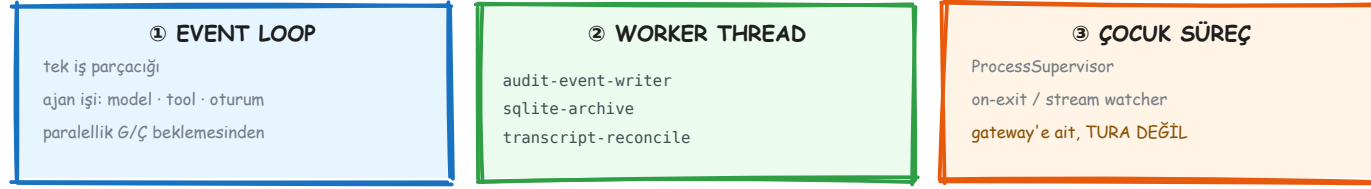
completed + parent_missing → blocked, FAILED DEĞİL

"Bu, tamamlanmış sonucu korur; çocuk yürütmesini yanlışlıkla başarısız diye raporlamak yerine."

status ≠ deliveryStatus ≠ terminalOutcome

Şemada üç ayrı enum var: işin *yürütülmesi*, sonucun *teslimatı*, ve işin *nihai sonucu*. Üçü ayrı tip olduğu için karıştırmak **mümkün değil** — tip sistemi hatırlatıyor. Somut senaryo: bir alt-ajan işini bitirdi, ama sonucu vereceği oturum kapanmıştı. Bu iş **blocked**, **failed değil**. Fark tamamen pratik. **failed** deseydin biri gelip işi yeniden koşturur, yani yapılmış işi ikinci kez yaptırırsın. **blocked** diyorsan iş zaten yapılmış, eksik olan yalnız teslimat. Çözüm de ona göre değişiyor: yeniden koşturmak değil, teslimatı düzeltmek.

Eşzamanlılık: üç katman



② neden var: cevap yolunda olmaması gereken defter işleri. Kuyruk dolarsa KAYIT düşer, KOŞU düşmez.

Bu bir geliştirici aracı için doğru öncelik — düzenlenmiş bir kurumda uyum hattında TERSİ olmalı.

event loop · worker thread · çocuk süreç

Node tek bir event loop'la çalışıyor, ama sistem tek katmanlı değil. **Ajan işi event loop'ta** koşuyor. Buradaki paralellik iş parçacığından gelmiyor, G/Ç beklemesinden geliyor: bir istek cevap beklerken başka iş yapılıyor. **Worker thread'ler**, cevap yolunda olmaması gereken defter işleri için: denetim kaydı yazmak, SQLite arşivlemek, transkript uzlaştırmak. Bunun doğrudan bir sonucu var — kuyruk dolarsa **kayıt düşüyor, koşu düşmüyor**. **Çocuk süreçler** uzun ömürlü izleyiciler için. Ve sahiplik kritik: izleyici gateway'e ait, tura değil. Tur bittiğinde izleyici ölmüyor.

Dayanıklı durum – ama durable execution değil

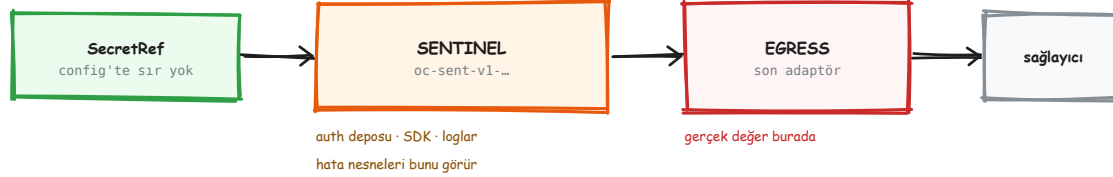
DURUM	DEPOLAMA	YENİDEN BAŞLATMADA
konuşma geçmişi	ajan SQLite	dokunulmaz
yarıda kalan tur	oturum satırı	otomatik devam
subagent koşulları	paylaşılan SQLite	boot'ta geri yüklenir
zamanlanmış işler	paylaşılan SQLite	tanım + geçmiş korunur
giden mesajlar	SQLite	drenaj edilir

- ① Gecikmiş işler YENİDEN ZAMANLANIR, anında oynatılmaz — soğuk başlangıçta yük tepesi olmasın.
 ② Kurtarma sınırlı: tekrar tekrar düşen oturum karantinaya alınır.
 ③ Uzlaştırma iki kanıtli: önce runtime, 5 dk grace sonrası dayanıklı koşu geçmişi.

kurtarma bir REPLAY değil, modele yazılan bir cümle

Her şey SQLite'ta duruyor: konuşma geçmişi, yarıda kalan tur, alt-ajan koşulları, zamanlanmış işler, kuyruktaki giden mesajlar. Yani süreç ölse de **durum kaybolmuyor**. Ama kurtarmanın *ne olduğu* önemli. Gateway, oturumu ajana geri veriyor ve yanına **sentetik bir sistem mesajı** ekliyor: "önceki turun kesildi, mevcut transkriptten devam et." Yani devam eden şey bir fonksiyon değil, bir **istem**. Deterministik replay yok. Tamamlanmış adımların memoizasyonu yok. Sonucu şu: bir tur yan etkili bir tool'u çağırdıktan sonra çöktüyse, o tool'un ikinci kez çağrılmasını **mekanik olarak engelleyen hiçbir şey yok**. Tek koruma, modelin transkripti okuyup fark etmesi.

Sentinel: gerçek değer son sınırdadır



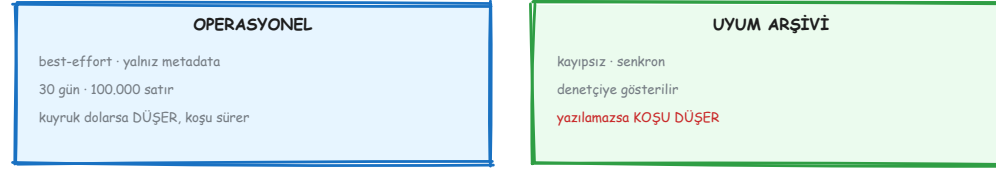
Bilinmeyen sentinel-gekilli değer **AĞ ETKİNLİĞİNDEN ÖNCE** fail-closed: çözülmemişse iletmektense istek gönderilmez.

Ve sınır beyanı: "Sentinel'ler süreç izolasyonu DEĞİLDİR" — gerçek değer aynı süreçte bellekte.

loglar sentinel görür, sağlayıcı gerçek değeri

Sırlar yapılandırmada düz metin olarak durmuyor. Yerlerine süreç-yerel, anlamsız bir **sentinel** yazılıyor. Auth deposu, SDK yapılandırması, loglar ve hata nesneleri hep *o sentinel'i* görüyor. Gerçek değer ancak istek süreçten çıkmadan hemen önce yerine konuyor. Ve tanınmayan, sentinel'e benzeyen bir değer görülürse istek **ağ etkinliğinden önce** durduruluyor. Çözülmemiş bir sentinel'i sağlayıcıya göndermektense hiç göndermemek tercih ediliyor. Kendi sınırını da yazmışlar, ve doğru yazmışlar: **bu bir süreç izolasyonu değil**. Gerçek değer aynı süreçte, bellekte duruyor.

İki kayıt hattı gerekiyor



OpenClaw yalnız solu yapıyor ve bunu açıkça söylüyor:

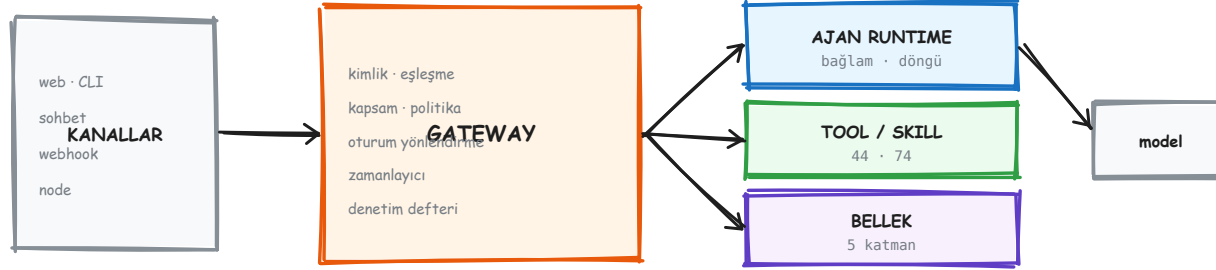
"Bir satırın yokluğu hiçbir şey kanıtlamaz."

KKB'nin ihtiyacı sağ taraf. Ayırımı baştan yapmak ucuz, sonradan şema göçü.

"Bir satırın yokluğu hiçbir şey kanıtlamaz."

OpenClaw'ın denetim kaydı **içerik tutmuyor**: prompt yok, tool argümanı yok, URL yok, komut çıktısı yok. Kimlikler de kurulum-yerel bir anahtarla takma ada çevriliyor. Kendi sınırını da açıkça yazıyorlar: *"Bu korelasyondur, anonimleştirme değildir."* Ve kayıt **best-effort**: kuyruk dolarsa satır düşüyor, koşu devam ediyor. Bir geliştirici aracı için bu doğru öncelik — kayıt uğruna işi durdurmak saçma olurdu. Düzenlenmiş bir kurumda ise önceliğin **tam tersi** gerekiyor: uyum hattı kayıpsız, senkron ve fail-closed olmalı. Yazılamıyorsa koşu düşmeli.

Geriye kalan tek cümle



Her şey Gateway'den geçiyor: kimlik, yetki, kayıt ve zamanlama tek yerde. Ajan runtime bunların hiçbirini bilmiyor.

mekanizma taşınır, güven modeli taşınmaz

Bu destedeki her mekanizma bir kurumsal asistana taşınabilir. Kapı, donmuş plan, köken sınıfı, iki kayıt hattı — hepsi. Taşınamayan tek şey şu: OpenClaw **tek bir güvenilen operatörün** etrafında tasarlanmış. Belgelerindeki bütün "bu bir güvenlik sınırı değildir" cümleleri buradan geliyor. O modelde zaten herkes güvenilir, dolayısıyla ayrımlar bir kolaylıktan ibaret. Birbirine güvenmeyen departmanların olduğu bir kurumda ise aynı cümleler birer **açık** haline geliyor. Yapılacak iş net: mekanizmaları al, **güven modelini yeniden kur**.